1주차_1,2장

1장. 머신 러닝과 딥러닝

1.1 인공지능, 머신 러닝과 딥러닝

구분	머신 러닝	딥러닝
동작 원리	입력 데이터에 알고리즘을 적용하여 예측을 수행한다.	정보를 전달하는 신경망을 사용하여 데이터 특징 및 관계를 해석한다.
재사용	입력 데이터를 분석하기 위해 다양한 알고리즘을 사용하며, 동일한 유형의 데이터 분석을 위한 재사용은 불가능하다.	구현된 알고리즘은 동일한 유형의 데이터를 분석하는 데 재사용된다.
데이터	일반적으로 수천 개의 데이터가 필요하다.	수백만 개 이상의 데이터가 필요하다.
훈련 시간	단시간	장시간
결과	일반적으로 점수 또는 분류 등 숫자 값	출력은 점수, 텍스트, 소리 등 어떤 것이든 가능

- 인공지능: 인간의 지능을 모방해 사람이 하는 일을 기계가 할 수 있게 하는 기술 → 머신 러닝 / 딥러닝으로 구현
- 머신러닝: 데이터를 인간이 전처리, 데이터의 특징 스스로 추출 X
- 딥러닝: 전처리 생략, 컴퓨터가 스스로 특성 추출

⇒ 인공지능 > 머신 러닝 > 딥러닝

1.2 머신 러닝이란

- 머신 러닝 학습 과정은 학습 단계와 예측 단계로 이루어짐

> 특성 추출: 데이터별 특징을 토대로 데이터를 벡터로 변환하는 것

- 머신러닝의 주요 구성 요소는 데이터와 모델
  - 데이터로 학습 모델 생성함 → 편향되지 않는 훈련 데이터 확보 필요

    - 모델은 학습 단계의 최종 결과물. 가설이라고도 함.
- 학습 절차: 모델 선택 → 모델 학습 및 평가 → 평가를 바탕으로 모델 업데이트
- 머신 러닝의 학습 알고리즘
    - 지도 학습: 정답(레이블)을 알려주고 학습
    - 비지도 학습: 정답을 알려주지 않고 데이터를 범주화해 예측
    - 강화 학습: 행동에 대한 보상을 받으며 학습

1.3 딥러닝이란

- 딥러닝: 심층 신경망 이론 기반의 머신 러닝 방법
    - 인간의 뇌의 뉴런과 시냅스 개념을 컴퓨터에 적용한 것
- 딥러닝 학습 과정
    1. 데이터 준비
    2. 모델 정의 - 신경망 생성
    3. 모델 컴파일 - 활성화 함수, 손실 함수, 옵티마이저 선택
    4. 모델 훈련 - 한 번에 처리할 데이터양 지정 (배치, 에포크)
    5. 모델 예측
- 딥러닝의 핵심 구성 요소는 신경망과 역전파
    - 심층 신경망(은닉층이 2개 이상) 사용 → 어떤 특성이 중요한지 스스로 학습
    - 오차를 이용해 역전파로 가중치 값 업데이트 ⇒ 파이토치가 자동으로 계산
- 딥러닝의 학습 알고리즘
    - 지도 학습
        - 이미지 분류, 인식, 분할 - 합성곱 신경망 (CNN)
        - 시계열 데이터 분석 - 순환 신경망 (RNN) / LSTM
    - 비지도 학습
        - 자연어 처리 - 워드 임베딩 (워드투벡터/글로브)
        - 데이터 분류 - 군집

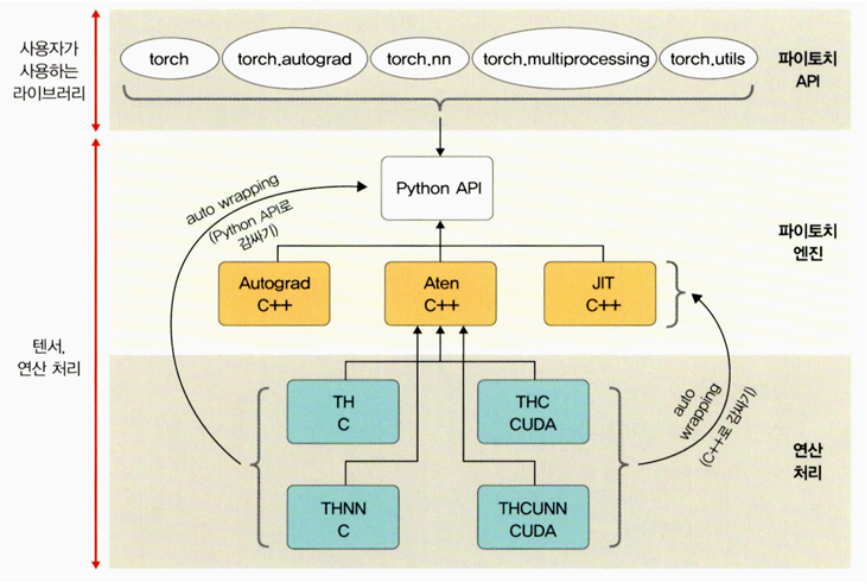
- 전이 학습: 사전 학습 모델 (ex. VGG, 인셉션, MobileNet)을 미세 조정 기법을 이용해 학습시키는 것
- 강화 학습

2장. 실습 환경 설정과 파이토치 기초

2.1 파이토치 개요

- 파이토치: GPU에서 텐서 조작 및 동적 신경망 구축이 가능한 딥러닝 프레임워크
  - 텐서: 파이토치의 데이터 형태 (다차원 배열)
  - 동적 신경망: 훈련 마다 네트워크 변경이 가능한 신경망
    - Define by Run 방식 (연산 그래프 정의 시 값도 초기화)

> 연산 그래프: 변수와 연산을 그래프로 표현하여 딥러닝에서 계산과 미분을 효율적으로 수행하게 해주는 구조



- 파이토치의 아키텍처
  - API → 엔진 → 연산 처리
- API: 사용자 인터페이스 제공, 엔진으로 작업 전달
  - `torch`: GPU 지원하는 텐서 패키지

    - `torch.autograd`: 자동 미분 패키지
    - `torch.nn`: 신경망 구축 및 훈련 패키지
    - `torch.multiprocessing`: 파이썬 멀티프로세싱 패키지
    - `torch.utils`: DataLoader 및 기타 유틸리티를 제공하는 패키지
- 엔진
    - Autograd C++: 가중치, 바이어스 업데이트 과정에서 미분 자동 계산
    - Aten C++: C++ 텐서 라이브러리 제공
    - JIT C++: 계산을 최적화하기 위한 JIT 컴파일러
    - Python API
- 연산 처리
    - C / CUDA 패키지: 거의 모든 계산 수행

> 텐서는 메모리에 1차원으로 저장됨 ⇒ 이를 스토리지라고 함.
>
> - 오프셋: 스토리지에서 텐서 시작 인덱스
> - 스트라이드: (row 이동 시 몇 칸 이동, col 이동 시 몇 칸 이동)
>
> ⇒ 전치 시키면 스토리지는 그대로이기 때문에 오프셋과 스트라이드로 구분

2.2 파이토치 기초 문법

[텐서 다루기]

- 텐서 생성: `torch.tensor([__])`
    - `device='cuda:0'` : gpu에 텐서 생성
    - `dtype=torch.float64`: dtype 설정
- 텐서를 ndarray로 변환: `__.numpy()`
- gpu상의 텐서를 cpu의 텐서로 변환: `__.to('cpu')`
- 인덱스 혹은 슬라이스로 접근 가능
- 텐서 간의 타입이 같을 때 수학 연산 가능
- 텐서의 차원 변경 (≈reshape): `__.view(_,_)`

- stack / cat: 텐서 결합
- t / transpose: 차원 교환

[데이터 호출]

- csv 파일 불러오기: `pd.read_csv('__')`
- 커스텀 데이터셋 구현:

```
import pandas as pd
import torch
from torch.utils.data import Dataset
from torch.utils.data import DataLoader

class CustomDataset(Dataset):
    def __init__(self, csv_file):
        self.label = pd.read_csv(csv_file)
        # 필요한 변수 선언, 데이터셋 전처리 함수

    def __len__(self):
        return len(self.label)
        # 총 샘플 수 가져오는 함수

    def __getitem__(self, idx):
        sample = torch.tensor(self.label.iloc[idx, 0:
3]).int()
        label = torch.tensor(self.label.iloc[idx, 3]).in
t()
        return sample, label
        # idx번째 데이터 반환하는 함수


tensor_dataset = CustomDataset('../covtype.csv')
dataset = DataLoader(tensor_dataset, batch_size=4, shuff
le=True)
```

⇒ 데이터를 조금씩 나누어 불러서 사용

- 파이토치에서 제공하는 데이터셋 사용:

```
pip install requests
import torchvision.transforms as transforms

mnist_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (1.0,))
])  # 평균이 0.5, 표준편차가 1.0이 되도록 데이터의 분포를 조정

from torchvision.datasets import MNIST
import requests

download_root = '../chap02/data/MNIST_DATASET' # 내려받을 경로 지정

train_dataset = MNIST(download_root, transform=mnist_transform, train=True,
                      download=True)  # training 데이터셋

valid_dataset = MNIST(download_root, transform=mnist_transform, train=False,
                      download=True)  # validation 데이터셋

test_dataset = MNIST(download_root, transform=mnist_transform, train=False,
                     download=True)  # test 데이터셋
```

[모델 정의]

- 모델을 정의하기 위해서 모듈을 상속한 클래스를 사용.

> - 계층(layer): 모듈을 구성하는 한 개의 계층 (ex. 합성곱층, 선형계층)
> - 모듈(module): 계층이 모여서 구성된 것
> - 모델(model): 최종적으로 원하는 네트워크

- 단순 신경망 정의 (모듈 상속 X): `model=nn.Linear(in_features=1,out_features=1, bias=True)`
- nn.Module 상속하여 정의:

```
class MLP(Module):

    def __init__(self, inputs):
        super(MLP, self).__init__()
        self.layer = Linear(inputs, 1)   # 계층 정의
        self.activation = Sigmoid()     # 활성화 함수 정의

    def forward(self, X):     # 모델에서 실행되는 연산 정의
        X = self.layer(X)
        X = self.activation(X)
        return X
```

- Sequential 신경망 정의 → 네트워크 모델들을 정의해주고 각 모듈을 순차적으로 실행해줌
- 함수로 신경망 정의 → 계층 재사용 가능하지만 모델이 복잡해짐

[파라미터 정의]

- 손실 함수: 학습 시 출력과 정답 사이의 오차를 측정.
  - BCELoss: 이진 분류에서 사용
  - CrossEntropyLoss: 다중 클래스 분류에서 사용
  - MSELoss: 회귀 모델에서 사용
- 옵티마이저: 데이터와 손실함수를 바탕으로 모델의 업데이트 방법 결정
  - `step()` 메서드로 전달받은 파라미터를 업데이트
  - 파라미터별로 다른 기준 적용 가능
  - `torch.optim.Optimizer(params,defaults)` : 모든 옵티마이저의 기본인 클래스
  - `zero_grad()` : 파라미터의 기울기를 0으로 만듦
- 학습률 스케줄러: 지정한 횟수의 에포크를 지날 때마나 학습률 감소 시켜줌.

  → 초기에는 빠른 학습, 전역 최소점 근처에서는 학습률을 줄임.
  - `torch.optim.lr_scheduler`
- 지표: 훈련과 테스트 단계를 모니터링

[모델 훈련]

1. `optimizer.zero_grad()` : 기울기 초기화

2. `output=model(input)` : 출력 계산
3. `loss=loss_fn(output,target)` : 오차 계산
4. `loss.backward()` : 역전파 학습 → 기울기 자동 계산
5. `optimizer.step()` : 기울기 업데이트

⇒ 에포크만큼 반복

[모델 평가]

- 함수로 모델 평가: `torchmetrics.functional.accuracy(preds, target)`
- 모듈로 모델 평가 :

```
import torch
import torchmetrics

metric = torchmetrics.Accuracy()  # 정확도 초기화

n_batches = 10
for i in range(n_batches):
    preds = torch.randn(10, 5).softmax(dim=-1)
    target = torch.randint(5, (10,))

    acc = metric(preds, target)
    print(f"Accuracy on batch {i}: {acc}")  # 현재 배치에서 정확도

acc = metric.compute()
print(f"Accuracy on all data: {acc}")  # 모든 배치에서 정확도
```

[훈련 과정 모니터링]

텐서보드로 파라미터 값의 변화를 시각화할 수 있음

1. 텐서보드를 설정
2. 텐서보드에 기록
3. 텐서보드를 사용해 모델 구조 살펴봄

```
import torch
from torch.utils.tensorboard import SummaryWriter

writer=SummaryWriter('저장될 위치')

for epoch in range(num_epochs):
        ...
        writer.add_scalar('Loss',loss,epoch)
        ...

writer.close()

# 텐서보드 실행
!tensorboard --logdir ../chap02/tensorboard --port 600
```

> - model.train(): 모델을 훈련 모드로 설정 (dropout 활성화)
> - model.eval(): 모델을 평가 모드로 설정 (dropout 비활성화)
> - torch.no_grad(): gradient 계산을 하지 않아 메모리와 연산 시간 절약

2.4 파이토치 코드 맛보기

- 범주형 데이터를 텐서로 변환:
  1. `dataset[category] = dataset[category].astype('category')`
  2. `category=dataset[category].cat.codes.values` → 범주형 데이터를 숫자로 변환
  3. `categorical_data=np.stack([ … ], 1)` → 넘파이 객체 합침
  4. `categorical_data=torch.tensor(categorical_data,dtype=torch.int64)`

  > - np.stack: 배열을 새로운 축으로 합침 → 차원이 동일해야 함
  > - np.concatenate(axis=): 선택한 축을 기준으로 연결
  > - ravel() / reshape(-1) / flatten() : 텐서의 차원을 1차원으로 변경

- 정확도: 전체 예측 건수에서 정답인 비율

$$\frac{True\ Positive + True\ Negative}{True\ Positive + True\ Negative + False\ Positive + False\ Negative}$$

- 재현율: 참인 것 중 참이라고 예측한 비율

$$\frac{True\ Positive}{True\ Positive + False\ Negative}$$

- 정밀도: 참이라고 예측한 것 중 실제로 참인 비율

$$\frac{True\ Positive}{True\ Positive + False\ Positive}$$

- F1-score: 정밀도와 재현율의 조화 평균

$$2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$